Lecture 7: parallelism

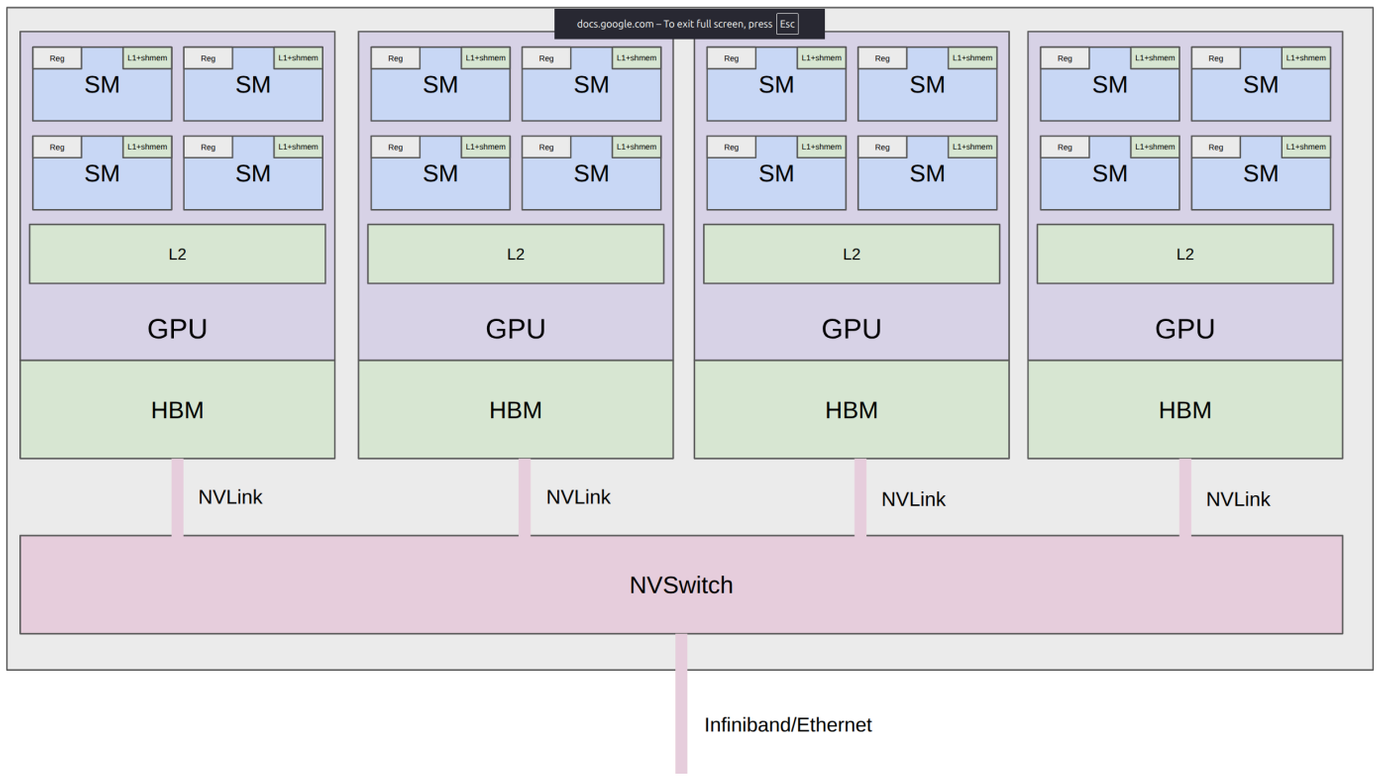
> 第 7 讲：并行

Last week: parallelism within a single GPU

> 上周：单 GPU 内部的并行

This week: parallelism across multiple GPUs

> 本周：跨多个 GPU 的并行



In both cases, **compute** (arithmetic logic units) is far from inputs/outputs (**data**).

> 这两种情况下，计算单元（算术逻辑单元）都远离输入/输出（数据）。

Unifying theme: orchestrate computation to avoid data transfer bottlenecks

> 统一主题：编排计算以规避数据搬运瓶颈

Generalized hierarchy:

> 广义的层级结构：

- Single node, single GPU: L1 cache / shared memory (fastest)

> 单节点、单 GPU：L1 缓存 / 共享内存（最快）

- Single node, single GPU: HBM

> 单节点、单 GPU：HBM

- Single node, multi-GPU: NVLink/NVSwitch

> 单节点、多 GPU：NVLink/NVSwitch

- Multi-node, multi-GPU: Infiniband/Ethernet (slowest)

> 多节点、多 GPU：Infiniband/以太网（最慢）

Last week: reduce memory accesses via fusion/tiling

> 上周：通过融合/分块（tiling）减少内存访问

This week: reduce communication across GPUs/nodes via replication/sharding

> 本周：通过复制/分片减少跨 GPU/节点的通信

Why do multi-GPU?

> 为什么要用多 GPU?

1. Your parameters (optimizer state + gradients + activations) don't fit on a single GPU.

> 你的参数（优化器状态 + 梯度 + 激活值）放不进单张 GPU。

2. You want to use more GPUs (more FLOPs) to train faster.

> 你想用更多 GPU（更多 FLOPs）来加快训练。

[stdout for this lecture]

> [本讲的 stdout]

Part 1: building blocks of distributed communication/computation

> 第一部分：分布式通信/计算的基本构件

Part 2: distributed training

> 第二部分：分布式训练

Walk through bare-bones implementations of each strategy on deep MLPs.

> 在深度 MLP 上逐个走读每种策略的最简实现。

Recall that MLPs are the compute bottleneck in Transformers, so this is representative.

> 回顾：MLP 是 Transformer 中的计算瓶颈，因此这具有代表性。

What's missing?

> 缺了什么?

- Communication/computation overlap

> 通信/计算重叠

- More general models (with attention, etc.)

> 更一般的模型（含注意力等）

- Other forms of parallelism (e.g., sequence parallelism, expert parallelism, combinations)

> 其他形式的并行（如序列并行、专家并行及其组合）

- Jax/TPUs: just define the model, the sharding strategy, and the Jax compiler handles the rest [levanter]

> Jax/TPU：只需定义模型和分片策略，其余交给 Jax 编译器 [levanter]

- But we're doing PyTorch so you can see how one builds up from the primitives

> 但我们用 PyTorch，这样你能看到如何从这些原语一步步搭建起来

Summary

> 总结

- Many ways to parallelize: data (batch), tensor/expert (width), pipeline (depth), sequence (length)

> 并行方式有多种：数据并行（批次维度）、张量/专家并行（宽度）、流水线并行（深度）、序列并行（长度）

- Data parallelism: DDP (all-reduce), FSDP/ZeRO (all-gather + reduce-scatter)

> 数据并行：DDP（all-reduce）、FSDP/ZeRO（all-gather + reduce-scatter）

- Tensor parallelism: requires very fast interconnects (e.g., NVLink)

> 张量并行：需要极快的互连（如 NVLink）

- Pipeline parallelism: can work with slow interconnects, but need to work to reduce pipeline bubbles

> 流水线并行：可以在慢速互连下工作，但需要设法减少流水线气泡

- Can **re-compute** or store in **memory** or store in another GPUs memory and **communicate**

> 可以重新计算、存放在内存中，或存到另一块 GPU 的内存中再通信

- Hardware is getting faster, but will always want bigger models, so will have this hierarchical structure

> 硬件越来越快，但我们总想要更大的模型，所以这种层级结构会一直存在

Collective operations are the conceptual primitives used for distributed programming [article]

> 集合通信操作是分布式编程所使用的概念原语 [文章]

- These are classic in the parallel programming literature from the 1980s.

> 它们在 20 世纪 80 年代的并行编程文献中已是经典。

- *Collective* means that you specify a general communication pattern across many devices.

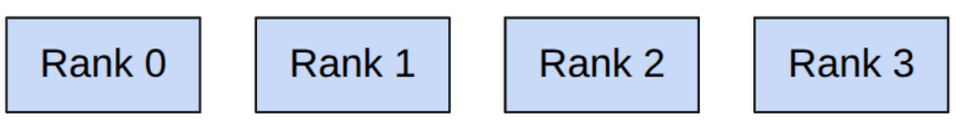
> 集合意味着你指定的是跨多台设备的通用通信模式。

- This can be better/faster than managing point-to-point communication yourself.

> 这可能比自己管理点对点通信更好/更快。

Setup:

> 设定：



- **Rank**: a particular device/GPU (e.g., 0, 1, 2, 3)

> Rank：某个具体的设备/GPU（如 0、1、2、3）

- **World size**: total number of devices (e.g., 4)

> World size：设备总数（如 4）

Operations:

> 操作：

- Broadcast, scatter, gather, reduce (foundations)

> Broadcast、scatter、gather、reduce（基础）

- All-gather, reduce-scatter, all-reduce (workhorse)

> All-gather、reduce-scatter、all-reduce（主力）

- All-to-all (for MoEs)

> All-to-all（用于 MoE）

Broadcast: copy from rank 0 to all ranks

> Broadcast：从 rank 0 拷贝到所有 rank

Minor use case: rank 0 loads initial checkpoint and broadcasts to all ranks

> 次要用例：rank 0 加载初始 checkpoint 并广播给所有 rank

Scatter tensor on rank 0 to all ranks

> 把 rank 0 上的张量 scatter 到所有 rank

Note: stepping stone to understanding reduce-scatter

> 注：这是理解 reduce-scatter 的垫脚石

Gather pieces from all ranks to rank 0 (opposite of scatter)

> 把所有 rank 的分片 gather 到 rank 0（scatter 的逆操作）

Note: stepping stone to understanding all-gather

> 注：这是理解 all-gather 的垫脚石

Reduce pieces from all ranks to rank 0, applying some operation (e.g., sum, min, max)

> 对来自所有 rank 的分片施加某种运算（如 sum、min、max）后 reduce 到 rank 0

Note: stepping stone to understanding all-reduce

> 注：这是理解 all-reduce 的垫脚石

All-gather: perform gather to all ranks, not just rank 0

> All-gather：gather 到所有 rank，而不只是 rank 0

Use case: each rank holds parameter shard, gather to get full parameters for forward pass

> 用例：每个 rank 持有参数分片，gather 得到完整参数用于前向传播

Reduce-scatter: perform reduce on each dimension, scatter results

> Reduce-scatter：在每个维度上执行 reduce，再 scatter 结果

Use case: after backward pass, sum gradients from different data shards, but distribute storage

> 用例：反向传播后，对不同数据分片的梯度求和，但分散存储

All-reduce = reduce-scatter + all-gather
Use case: after backward pass, sum gradients from different data shards, but replicate full parameters

> 用例：反向传播后，对不同数据分片的梯度求和，并复制完整参数

Breaking all-reduce into reduce-scatter + all-gather allows for flexibility (e.g., ZeRO/FSDP)

> 把 all-reduce 拆成 reduce-scatter + all-gather 可以带来灵活性（如 ZeRO/FSDP）

All-to-all: each rank sends each other rank some tensor (most general)

> All-to-all：每个 rank 向其他每个 rank 发送某个张量（最通用的形式）

Notes:

> 注：

- Useful for MoEs: each rank has split of data and subset of experts; need to route data to experts

> 对 MoE 很有用：每个 rank 持有数据的一个切分和一部分专家；需要把数据路由到专家

- For balanced splits, all-to-all looks like transpose

> 对均衡的切分，all-to-all 看起来就像转置

- Also handles unbalanced splits (but want splits to be as balanced as possible)

> 也能处理不均衡的切分（但应尽量让切分均衡）

Way to remember the terminology:

> 记忆这套术语的方法：

- Reduce: performs some associative/commutative operation (sum, min, max)

> Reduce：执行某种可结合/可交换的运算（sum、min、max）

- Scatter is inverse of gather

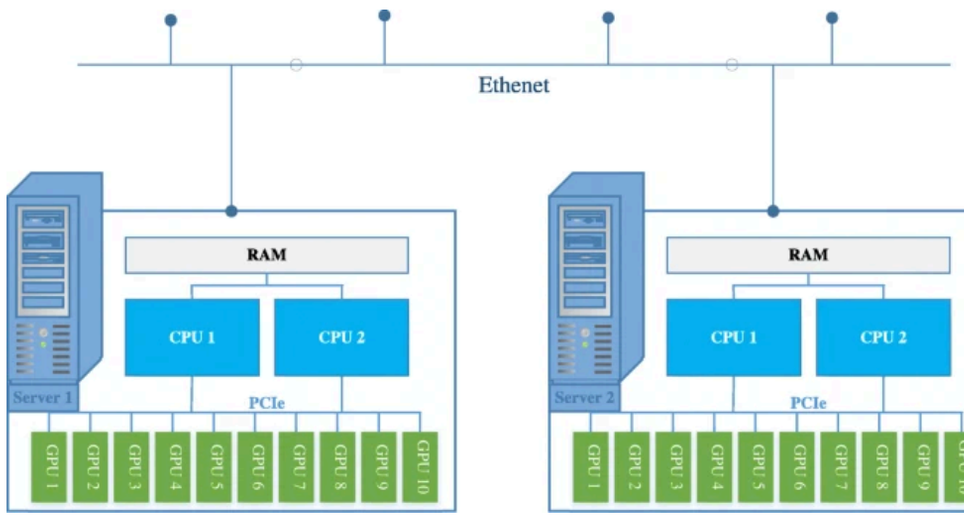
> Scatter 是 gather 的逆操作

- All: means destination is all devices

> All：意味着目标是所有设备

Classic (in the home):

> 经典场景（家庭环境）：



- GPUs on same node communicate via a PCI(e) bus (v7.0, 16 lanes => 242 GB/s) [article]

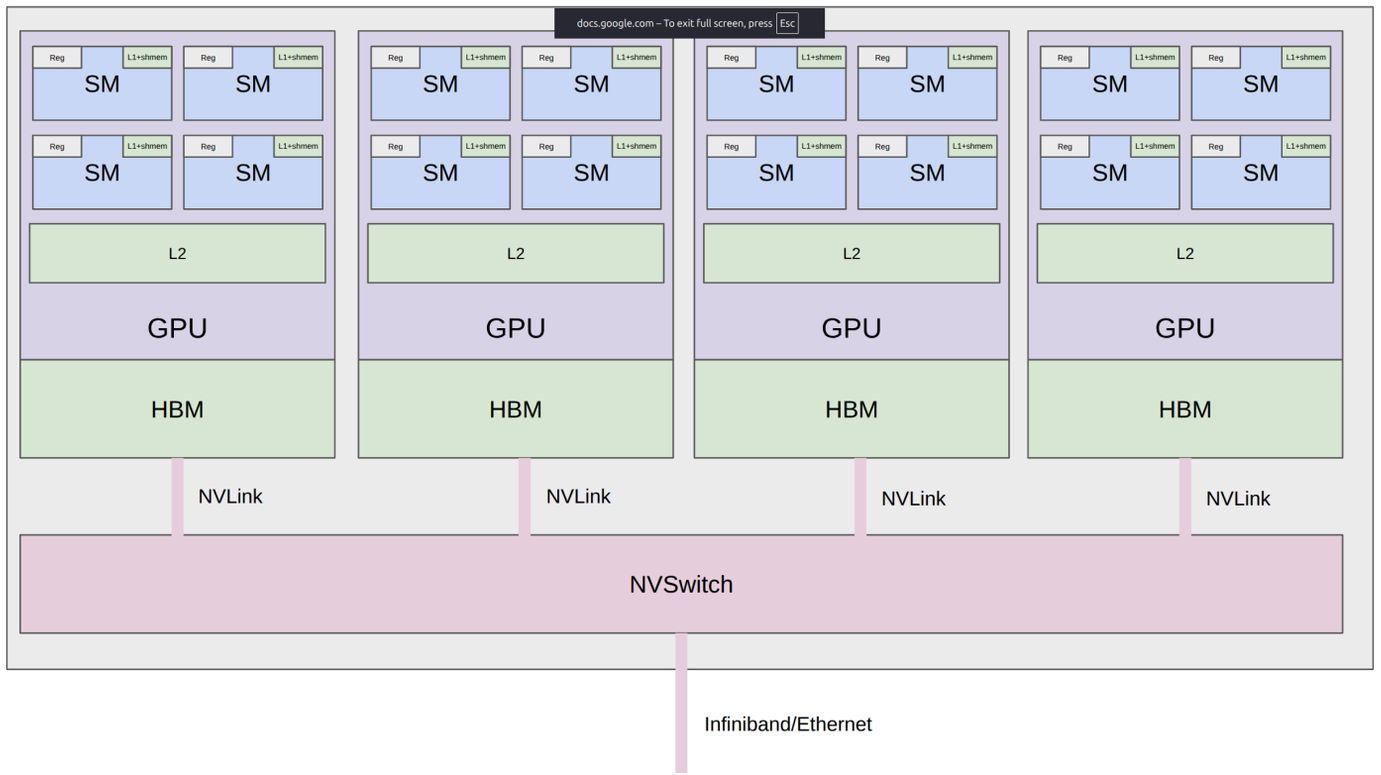
> 同一节点内的 GPU 通过 PCI(e) 总线通信（v7.0，16 条通道 => 242 GB/s） [文章]

- GPUs on different nodes communicate via Ethernet (~200 MB/s)

> 不同节点的 GPU 通过以太网通信（约 200 MB/s）

Modern (in the data center):

> 现代场景（数据中心）：



Typical setup:

> 典型配置：

- 8 GPUs per node, connected by NVLink to an NVSwitch (B200s' NVLink 5.0 gets 1.8 TB/s; HBM was 8 TB/s)

> 每节点 8 块 GPU，经 NVLink 连到 NVSwitch（B200 的 NVLink 5.0 达 1.8 TB/s；HBM 为 8 TB/s）

- 256 nodes per pod, connected by Infiniband (via PCIe -> HCA / Infiniband NIC -> Infiniband cable) (~0.05 TB/s)

> 每个 pod 256 个节点，经 Infiniband 连接（路径为 PCIe -> HCA / Infiniband 网卡 -> Infiniband 线缆）（约 0.05 TB/s）

- N pods per cluster / datacenter, connected by Ethernet (via PCIe -> CPU)

> 每个集群/数据中心 N 个 pod，经以太网连接（路径为 PCIe -> CPU）

Bypassing the CPU:

> 绕过 CPU：

- Ethernet requires passing through the CPU (copying data to kernel socket buffer, build TCP packets, copy to NIC ring buffer)

> 以太网需要经过 CPU（把数据拷贝到内核 socket 缓冲区、构建 TCP 包、拷贝到网卡环形缓冲区）

- Remote Direct Memory Access (RDMA): allows one GPU to directly read/write another GPU's memory without involving the CPU

> 远程直接内存访问（RDMA）：允许一块 GPU 直接读/写另一块 GPU 的内存，无需 CPU 介入

- Infiniband supports RDMA, but standard Ethernet does not

> Infiniband 支持 RDMA，而标准以太网不支持

Advancements:

> 新进展：

- GB200/GB300 NVL72: 8 GPUs per tray, 9 trays per rack -> 72 GPUs in one NVLink domain

> GB200/GB300 NVL72：每个托盘 8 块 GPU，每个机架 9 个托盘 -> 一个 NVLink 域内共 72 块 GPU

- RDMA over Converged Ethernet (RoCE): Ethernet bypasses CPU, similar but cheaper/weaker than Infiniband, used by Meta

> RDMA over Converged Ethernet（RoCE）：以太网绕过 CPU，与 Infiniband 类似但更便宜/性能更弱，Meta 在用

NVIDIA Collective Communication Library (NCCL)

> NVIDIA 集合通信库（NCCL）

NCCL translates collective operations into low-level packets that are sent between GPUs. [talk]

> NCCL 把集合通信操作翻译成在 GPU 之间发送的底层数据包。 [演讲]

- Detects topology of hardware (e.g., number of nodes, switches, NVLink/PCIe)

> 探测硬件拓扑（如节点数、交换机、NVLink/PCIe）

- Optimizes the path between GPUs

> 优化 GPU 之间的传输路径

- Launches GPU kernels to send/receive data

> 启动 GPU kernel 来收发数据

PyTorch distributed library (`torch.distributed`) [documentation]

> PyTorch 分布式库（torch.distributed） [文档]

- Provides clean interface for collective operations (e.g., `all_gather_into_tensor`)

> 为集合通信操作提供简洁的接口（如 all_gather_into_tensor）

- Supports multiple backends for different hardware: gloo (CPU), nccl (GPU)

> 支持多种后端以适配不同硬件：gloo（CPU）、nccl（GPU）

- Also supports higher-level algorithms (e.g., `FullyShardedDataParallel`) [not used in this course]

> 也支持更高层的算法（如 FullyShardedDataParallel）[本课程未使用]

Let's walk through some examples.

> 我们来看几个例子。

Indeed, all-reduce = reduce-scatter + all-gather!

> 没错，all-reduce = reduce-scatter + all-gather！

How fast does communication happen?

> 通信到底有多快？

References:

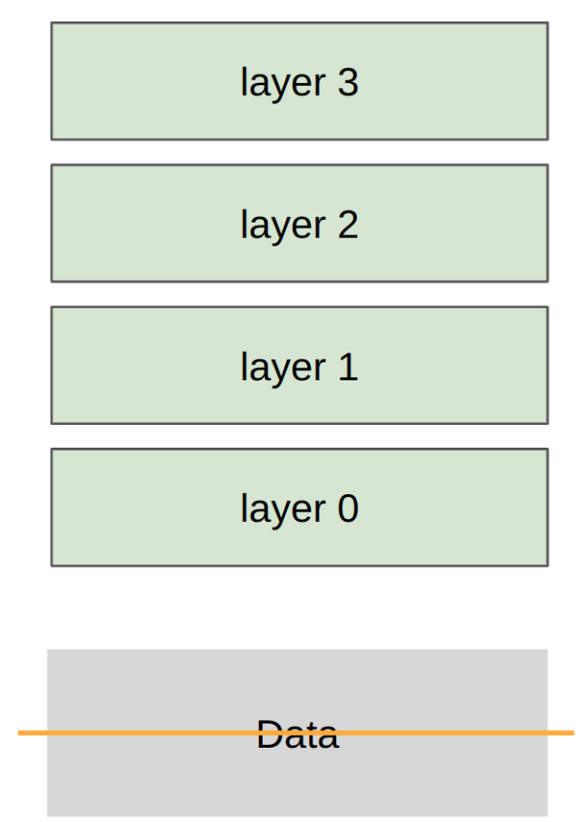
> 参考文献：

[How to reason about collective operations]

> [如何分析集合通信操作]

[Sample benchmarking code]

> [基准测试示例代码]



Sharding strategy: each rank gets a slice of the data

> 分片策略：每个 rank 拿到一部分数据

Notes:

> 注：

- Losses are different across ranks (computed on local data)

> 各 rank 的损失不同（在本地数据上计算）

- Gradients are all-reduced to be the same across ranks

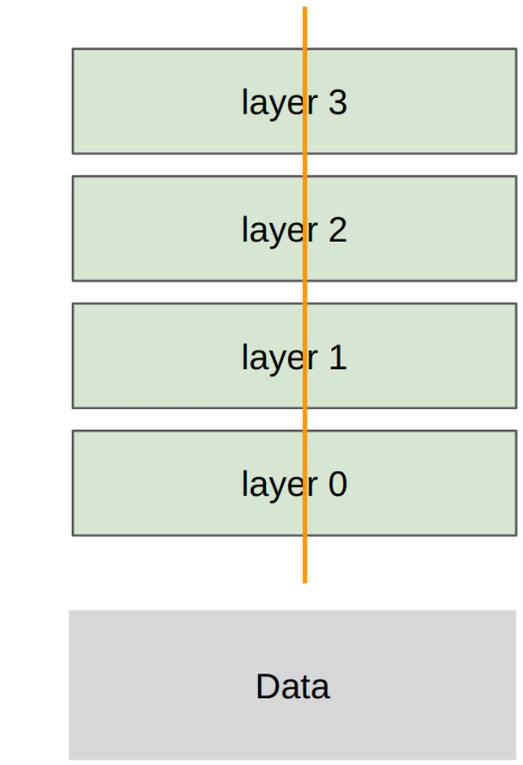
> 梯度经 all-reduce 后在各 rank 上一致

- Therefore, parameters remain the same across ranks

> 因此参数在各 rank 上保持一致

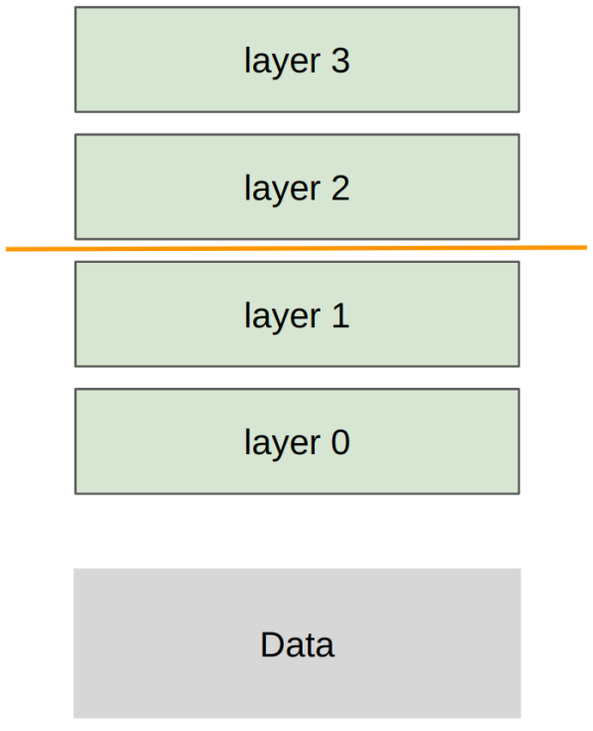
Next time: FSDP/ZeRO: use all-gather and reduce-scatter to avoid holding all parameters in memory

> 下次讲：FSDP/ZeRO：用 all-gather 和 reduce-scatter 避免把全部参数都留在内存中



Sharding strategy: each rank gets part of each layer, transfer all data/activations

> 分片策略：每个 rank 拿到每一层的一部分，传输全部数据/激活值



Sharding strategy: each rank gets subset of layers, transfer all data/activations

> 分片策略：每个 rank 拿到一部分层，传输全部数据/激活值

Not handled: overlapping communication/computation to eliminate pipeline bubbles

> 未涉及：通过通信/计算重叠来消除流水线气泡